

Orchestrating Agentic Coding AIs: A Roadmap-Driven Protocol

DOT E 3010

Abstract

This document establishes a formal protocol for utilizing "Roadmap-Driven Development" (RDD) to guide Agentic AIs (such as Cursor/Composer) in building complex software systems. We analyze the interaction between human intent (the Roadmap) and machine execution (the Agent), specifically focusing on the epistemological boundaries of the AI—what it can perceive, modify, and influence versus the external system states that remain opaque to it. Using the *Solver#42* project as a case study, we demonstrate how to prevent "context drift" and manage the "Agentic Gap" between code generation and environmental execution.

Contents

Introduction: The Agentic Coding Paradigm	3
Part I: Roadmap-Driven Development (RDD)	3
The Roadmap as a Mutable State Machine	3
Step-by-Step Guidance Protocol	4
Context Loading	4
The Implementation Cycle	4
The Verification and Update Step	4
Handling Hallucinations and Corrections	4
Part II: The Epistemological Boundaries of the Agent	5
The Visible World: What the Agent Can See and Modify	5
The File System (Textual Representation)	5
The Terminal (Standard Output/Error)	5
Linter and Language Server Protocol (LSP)	5
The Invisible World: What the Agent Cannot See or Touch	6
GUI Applications and Daemons	6
Root Permissions and OS Security	6
Browser Visuals (rendered HTML/CSS)	6
System Resource State	7
Case Study: The Solver#42 Execution Strategy	7
Phase 1: The Blind Logic	7
Phase 2: The Docker Barrier	7
Phase 5: The Packaging Illusion	8
Conclusion: The Human as the Runtime Environment	8

Introduction: The Agentic Coding Paradigm

The emergence of Agentic AI in software development represents a fundamental shift from "autocomplete" to "autonomous execution." However, the efficiency of an AI Agent is not solely determined by the underlying model's intelligence (e.g., GPT-4o or Claude 3.5 Sonnet), but rather by the structure of the constraints provided by the human operator.

In a traditional workflow, a developer holds the architectural map in their mind. In an Agentic workflow, this map must be externalized into a persistent, machine-readable format. This document argues that a static Markdown file, specifically a `roadmap.md`, serves as the most effective "Context Anchor" for maintaining long-term project coherence.

We will explore two critical dimensions of this workflow:

1. **Teleological Guidance:** How to force the AI to adhere to a linear progression of tasks using a roadmap.
2. **Ontological Boundaries:** Defining strictly what the AI Agent interacts with (The IDE Context) versus the reality it cannot touch (The System Context).

Part I: Roadmap-Driven Development (RDD)

The Roadmap as a Mutable State Machine

Most developers treat roadmaps as documentation for humans. In Agentic Coding, the `roadmap.md` is a control plane. It acts as a shared memory state between the human and the AI.

A roadmap for an AI must not be vague. It must be structured as a dependency graph. As seen in the *Solver#42* project, the roadmap was divided into five distinct phases:

- Phase 1: Core Engine (Pure Python)
- Phase 2: Data Layer (Docker/SQL)
- Phase 3: API Service (FastAPI)
- Phase 4: Frontend (UI/UX)
- Phase 5: Packaging (Local Deployment)

This "Inside-Out" approach is critical. If an Agent is asked to build the UI before the database, it will hallucinate mock data structures that inevitably fail to match the final backend implementation.

Step-by-Step Guidance Protocol

To successfully guide the AI, the operator must enforce a strict "Read-Execute-Update" loop.

Context Loading

Before asking for a single line of code, the Agent must ingest the roadmap. The user must explicitly explicitly prompt:

"Read @roadmap.md. We are currently at the beginning of Phase 1. Do not attempt Phase 2. Acknowledge the goals of Phase 1."

This constrains the AI's search space. It prevents the model from optimizing for future problems (like Docker containerization) that confuse the immediate task (writing a Python class).

The Implementation Cycle

When implementing a specific feature (e.g., 'StandardAnswerGenerator'), the Agent should be instructed to check the roadmap item.

```
1 # User Prompt:
2 "Implement the StandardAnswerGenerator class as defined in Phase 1 of
   @roadmap.md.
3 Only use local Python libraries for now. Do not add database imports
   yet."
```

The Verification and Update Step

This is the most crucial step often missed. Once the code is written and tests pass, the Agent must be instructed to **update the roadmap itself**.

- **Why?** When the AI modifies 'roadmap.md' to mark a checkbox as '[x] Completed', it reinforces its own training context. The "memory" of the completion becomes part of the file history.
- **Prompt:** *"We have finished the CLI tool. Update @roadmap.md to mark Phase 1 as complete. Then, explain what Phase 2 entails based on the file."*

Handling Hallucinations and Corrections

The AI is fallible. It may claim a task is done when the code is buggy. **Rule of Thumb:** Never let the AI proceed to the next item on the roadmap until the current item is verified

by the human via a terminal run.

If the AI fails, the roadmap serves as a reset point. You can say: *"Revert the last change. Re-read Phase 2 of @roadmap.md. You missed the requirement for PostgreSQL."*

Part II: The Epistemological Boundaries of the Agent

A common failure mode in Agentic coding is assuming the Agent "knows" the state of your computer. It does not. The Agent operates within a strictly defined sandbox. We must distinguish between the **Visible World** (IDE Context) and the **Invisible World** (System Context).

The Visible World: What the Agent Can See and Modify

The Agent (Cursor/Composer) has access to a specific set of interfaces. It is "omnipotent" only within these boundaries.

The File System (Textual Representation)

The Agent sees files as strings of text.

- **Capability:** It can read '@main.py', analyze the syntax, and rewrite the entire file.
- **Limitation:** It cannot see the "file icon" or the folder color. It relies on file paths (e.g., '/backend/app/main.py'). If a file is not added to its context, it does not exist for the Agent.

The Terminal (Standard Output/Error)

The terminal is the Agent's only sensory organ for "Runtime Reality."

- **Capability:** If the Agent runs 'python main.py' and the terminal outputs 'Traceback (most recent call last)...', the Agent "sees" the error and can fix it.
- **Critical Workflow:** The user must allow the Agent to run terminal commands to "ground" its hallucinations. If the Agent writes code but never runs it, the code is likely broken.

Linter and Language Server Protocol (LSP)

The Agent receives signals from the IDE's linter (red squiggly lines).

- **Capability:** It can see that ‘import numpy’ is underlined because the package is missing.
- **Action:** It can propose ‘pip install numpy’.

The Invisible World: What the Agent Cannot See or Touch

The Agent is blind to anything outside the text editor’s window and the terminal’s text stream. This is where the human **MUST** intervene.

GUI Applications and Daemons

Scenario: The Agent tries to start a database using ‘docker-compose up’.

- **Agent’s View:** It sends the command to the terminal.
- **Terminal Output:** ‘Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?’
- **The Gap:** The Agent **cannot** see the Docker Desktop icon in the macOS menu bar. It cannot click the Docker icon to start the application.
- **Human Role:** The human must physically open Docker Desktop. The Agent can only wait.

Root Permissions and OS Security

Scenario: The Agent tries to listen on port 80.

- **Agent’s View:** ‘Permission denied’.
- **The Gap:** The Agent cannot enter the user’s ‘sudo’ password blindly (security risk), nor can it interact with macOS ”Allow Network Connection” pop-ups.
- **Human Role:** The human must type the password or click ”Allow”.

Browser Visuals (rendered HTML/CSS)

Scenario: The Agent writes HTML for the *Solver#42* frontend.

- **Agent’s View:** It sees ‘div class=’glass-panel’’. It ”thinks” this looks like glassmorphism because it knows the CSS properties.

- **The Gap:** It cannot see if the div is misaligned by 5 pixels or if the text color contrasts poorly with the background.
- **Human Role:** The user must open the browser, look at the page, and describe the visual bug back to the Agent (e.g., *"The button is overlapping the text box"*).

System Resource State

The Agent does not know if your battery is low, if your CPU is overheating, or if your WiFi is disconnected (unless a specific command fails with a network error).

Case Study: The Solver#42 Execution Strategy

In the development of *Solver#42*, we utilized this separation of concerns to maximize efficiency.

Phase 1: The Blind Logic

We asked the Agent to write `'standard_answer_generator.py'`.

- **Agent Action:** Wrote Python code.
- **Validation:** We asked the Agent to write a `'test_script.py'` and run it in the terminal.
- **Result:** The Agent saw the text output of the generated answer. Success was verified purely through text.

Phase 2: The Docker Barrier

We asked the Agent to set up PostgreSQL.

- **Agent Action:** Wrote `'docker-compose.yml'`.
- **Failure:** The Agent tried to run it, but Docker wasn't started.
- **Human Intervention:** User started Docker Desktop.
- **Agent Recovery:** User told Agent "Docker is now running. Retry." Agent ran the command again, successfully.

Phase 5: The Packaging Illusion

We asked the Agent to create a "One-Click Launcher" ("start_mvp.command").

- **Agent Action:** Wrote a shell script.
- **Blind Spot:** The Agent cannot double-click the script to test if the macOS Terminal window opens correctly.
- **Human Role:** The user had to manually run the script and report back: "It opened, but the Python virtual environment didn't activate."
- **Correction:** The Agent modified the script to use absolute paths based on the user's feedback.

Conclusion: The Human as the Runtime Environment

The most effective mental model for "Agentic Coding" is that the AI is the **Architect and the Typist**, but the Human is the **Runtime Environment and the Senses**.

The AI drafts the roadmap. The AI writes the code. The AI attempts to run the code. But the Human must:

1. **Validate** that the roadmap aligns with business goals.
2. **Provide** the sensory input (visual checks, OS status) that the Agent lacks.
3. **Unlock** the doors (permissions, external apps) that the Agent cannot open.

By strictly adhering to a file-based roadmap ('roadmap.md') and understanding these epistemological boundaries, developers can push Agentic AI to build complex, robust applications like *Solver#42* with minimal friction.